

Struktur Proyek

- Folder **contract** berisi kode smart contract (backend)
- Folder **frontend** berisi kode tampilan aplikasi (frontend)
- Folder **target** berisi hasil build dari smartcontract

```
▼ workshop-starter
  > 📁 contracts
  > 📁 frontend
  > 📁 target
  📁 .gitignore
  📁 Cargo.lock
  📁 Cargo.toml
  📄 README.md
```

Contract

```
#![no_std]
use soroban_sdk::{contract, contractimpl,

#[contract]
pub struct NotesContract;

#[contractimpl]
> impl NotesContract { ...
}
```

Struct

- Struct adalah tipe data yang digunakan untuk menyimpan beberapa data dalam satu objek
- Digunakan untuk mempresentasikan suatu objek
- Setiap data di dalam struct disebut field
- Di smart contract, struct sering digunakan untuk merepresentasikan data seperti user, note, mahasiswa dll.

Struct

```
#[contracttype]
#[derive(Clone, Debug)]
pub struct Note {
    id: u64,
    title: String,
    content: String,
}
```

Contract function

- Function adalah fitur di dalam smart contract untuk menjalankan suatu aksi
- User memanggil function untuk berinteraksi dengan contract
- Function bisa digunakan untuk mengambil atau mengubah data

Contract function

```
#[contractimpl]
impl NotesContract {
    // Fungsi untuk mendapatkan semua notes
    > pub fn get_notes(env: Env) -> Vec<Note> { ...
    }

    // Fungsi untuk membuat note baru
    > pub fn create_note(env: Env, title: String, content: String) -> String { ...
    }

    // Fungsi untuk menghapus notes berdasarkan id
    > pub fn delete_note(env: Env, id: u64) String { ...
    }
}
```


Storage

- Storage digunakan untuk menyimpan data contract di dalam blockchain
- Data disimpan dalam bentuk key-value dan diakses melalui `env.storage()`
- Tipe storage ada 2, ada get dan set.
- Get artinya untuk mengambil data
- Set artinya untuk menyimpan data

Storage

Key: "NOTE_DATA"

Value:

```
[  
  {title: "belajar rust", dibuat: "2023" },  
  {title: "memahami stellar", dibuat: "2024"},  
]
```



array

- Note 1
- Note 2
- ...

Storage

1. Buat key

```
const NOTE_DATA: Symbol = symbol_short!("NOTE_DATA");
```

2. Akses ke storage

```
env.storage().instance().get(KEY);
```

```
env.storage().instance().set(KEY, VALUE);
```

Create, Read, Delete

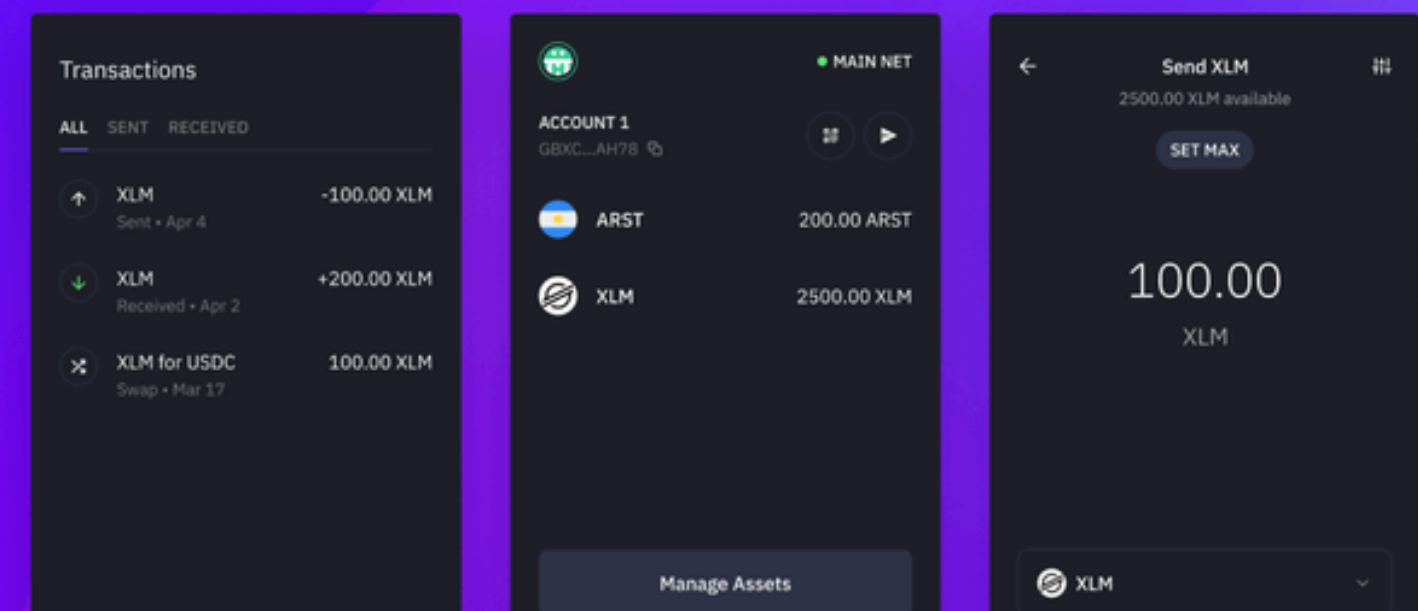
Deploy to Testnet

- Testnet adalah jaringan blockchain untuk testing tanpa menggunakan uang asli
- Digunakan untuk mencoba deploy dan menjalankan smart contract secara testing
- Semua aset di testnet tidak memiliki nilai nyata (hanya untuk simulasi)
- Untuk melakukan deploy, diharuskan untuk memiliki akun wallet dan memiliki token didalamnya

Wallet

- Digunakan untuk login dan berinteraksi dengan aplikasi Web3
- Setiap wallet memiliki address sebagai identitas di blockchain
- di Stellar ada 2 cara untuk membuat wallet, menggunakan terminal dan menggunakan aplikasi freighter wallet extension di browser

A Stellar wallet
for every website



Deploy to testnet

1. Membuat wallet

```
stellar keys generate indramahesa --network testnet --fund
```

2. Build Smart Contract

```
stellar contract build
```

3. Deploy

```
stellar contract deploy --source-account indramahesa
```


Memanggil smart contract

```
stellar contract invoke  
  --id ABCABCABC  
  --source-account indramahesa  
  -- get_notes
```


Submission

- Submission dikumpulkan sebelum workshop berakhir
- Gunakan bantuan AI untuk membantu pengerjaan
- Aplikasi harus berbeda dari contoh yang dibuat saat workshop (boleh modifikasi sederhana)
- Sertakan README yang jelas (deskripsi aplikasi, fitur, ID smartcontract testnet, screenshot tampilan testnet, bahasa inggris)
- Nama github harus sesuai dengan konteks aplikasi
- Submission bersifat wajib untuk mendapatkan sertifikat